Planetary Rover — Solar Energy and Charging Dynamics

Hani Bouhraoua 8314923

Assignment D1-V7 | AI4Ro2 — Artificial Intelligence for Robotics 2 |}

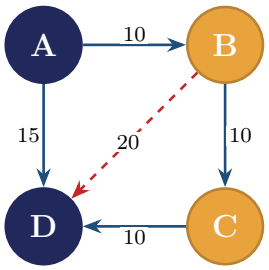
Planner: ENHSP. Q1 = classical PDDL · Q2 = PDDL+ (processes & events).

1. Domain and Planning Problem

A single solar-powered rover must explore a graph of locations and **visit every node without depleting its battery**. Moving consumes energy; two sun-exposed zones recharge it, with efficiency that varies by location. The rover must *balance exploration against energy recovery.*

Map (directional). The scenic route `A→B→C→D` (cost 10 each) visits everything. `A→D` (15) skips B and C; `B→D` (20) is a decoy that skips C. No edge enters A, so the rover starts there. Solar zones: **B** and **C**.

Objects vs. properties. Only two object types exist: `rover` and `location`. Battery, sunlight and time are *not* objects — they are properties or quantities of the world, so each becomes a predicate or a numeric fluent (see §2).



• B, C = solar zones
B→D = decoy (skips C)
Goal: visit A, B, C, D.

2. Modelling Choices

Predicates: `at`, `connected`, `solar-zone`, `visited`, and (Q2) `daytime`.
Numeric fluents: `battery-level`, `max-battery`, `move-cost`, `charge-rate`, and (Q2) `time-of-day`.

- **Energy as a hard constraint.** `move` requires `(>= (battery-level ?r) (move-cost ?from ?to))` and decreases the battery on use, so the rover cannot cross an edge it cannot afford. `move-cost` (consumption) is deliberately separate from `charge-rate` (generation), and `charge-rate` is per-location to capture varying efficiency.
- **Q1 — recharge is an *action*.** A discrete `(:action recharge)` adds `charge-rate` instantly at a solar zone. The instantaneous abstraction is acceptable in classical PDDL (no time) and is justified because Q2 replaces it with a continuous process.
- **Q2 — recharge is a *process*.** `(:process charging)` raises the battery by `(* #t (charge-rate ?l))` — a continuous, rate-based gain integrated over the time spent stationary at a solar zone in daytime. The rover does not *choose* to charge; the world does it automatically while the conditions hold.
- **Q2 — nightfall is an *event*.** `(:event nightfall)` fires the instant `(>= (time-of-day) 100)` and sets `(daytime)` false. A `(:process time-flow)` advances the clock, and the goal carries `(< (time-of-day) 100)` so the mission must finish before nightfall.
- **How they interact.** The `charging` process and the `nightfall` event are *not* linked explicitly — they coordinate through the shared `(daytime)` fact. When nightfall flips it off, charging's precondition fails and it stops the same instant. Charge rates are rescaled from Q1's one-shot jumps (15, 25) to per-tick rates (0.2, 0.5), because in Q2 they multiply by elapsed time.

3. Plan Output and Walkthrough

All results are the real ENHSP outputs (full logs in `codes/outputs/`); each matches the hand-traced prediction.

Q1 — recharge optional vs. necessary

The two problems differ *only* in starting battery. With 50 the rover finishes on three moves and never recharges; with 25 it is forced to recharge at C, else it is stranded there with 5 units (< the 10 needed for C→D).

Optional (battery 50) — Plan-Length 3:

```
0.0: (move rover1 locA locB)
1.0: (move rover1 locB locC)
2.0: (move rover1 locC locD)
```

Necessary (battery 25) — Plan-Length 4:

```
0.0: (move rover1 locA locB)
1.0: (move rover1 locB locC)
2.0: (recharge rover1 locC)
3.0: (move rover1 locC locD)
```

Q2 — timing decides feasibility

The four cases share the same map, rates and goal; only the starting `time-of-day` changes. Charging appears as `---waiting--` gaps (the rover sitting in the sun), not as an explicit action.

Case	Battery / start	Elapsed	Outcome
`easy`	60 / t=0	0	3 moves, no charging — process unused
`tight`	20 / t=0	47	charges B+C; *valid but not time-optimal*
`fastzone`	20 / t=79	20	forced into the *optimal* plan (C only)
`infeasible`	20 / t=85	—	unsolvable — nightfall blocks the goal

tight (t=0). With plenty of daylight, the satisficing search takes the first valid plan: 45 ticks at the *slow* zone B (+9) and 2 at C (+1), recovering the needed 10 but taking 47 ticks — valid, not optimal.

```
0:    (move rover1 locA locB)
0:    -----waiting---- [45.0]      ; charging at B (rate 0.2)
45.0: (move rover1 locB locC)
45.0: -----waiting---- [47.0]      ; charging at C (rate 0.5)
47.0: (move rover1 locC locD)      ; Elapsed Time: 47.0
```

fastzone (t=79). Only 21 ticks of daylight remain, so the slow-zone strategy no longer fits. The planner is forced into the optimal plan: charge exactly 20 ticks at the fast zone C and arrive at t=99 — one tick before nightfall.

```
0:    (move rover1 locA locB)
0:    (move rover1 locB locC)
0:    -----waiting---- [20.0]      ; charging at C only (rate 0.5)
20.0: (move rover1 locC locD)      ; Elapsed Time: 20.0
```

infeasible (t=85). Only 15 ticks of daylight remain, but the minimum charge needed is $10/0.5 = 20$ ticks. Nightfall stops charging too soon: ENHSP explores 815 states, hits 151 dead-ends, and reports *Problem unsolvable* — the correct, expected result.

4. Discussion

Discrete vs. continuous. Q1's instantaneous recharge gives short, easily verified plans but cannot say *how long* the rover must sit. Q2's process answers exactly that and exposes the waiting gaps a real controller needs. It also changes *agency*: in Q1 the planner is the only cause of change, while in Q2 the world acts on its own through processes and events.

Validity, not optimality. The `tight` (47) and `fastzone` (20) cases differ only in the deadline, yet the first is far from optimal. The `hadd` heuristic counts remaining *actions*, not *time*, so the satisficing search stops at the first valid plan; only a tighter deadline (or a `(:metric)`) forces optimality.

Uncertainty. The model assumes a fully known, deterministic world. Real rovers have noisy sensors and variable conditions, which PDDL+ cannot express (a fluent is one number, not a distribution). The right tools are **MDPs** (uncertain outcomes), **POMDPs** (noisy sensing → belief states), or **robust** planning (worst-case values). These were not used: the task targets PDDL/PDDL+, no mainstream PDDL planner supports probabilities, and the deterministic model already shows the core lesson. In practice it would sit inside a *Sense–Plan–Act* loop that re-plans from observed state.

5. Limitations

- **Optimality.** Satisficing plans can be time-wasteful (the 47-tick `tight` case); a metric or tighter constraints are needed for optimality.
- **Instantaneous moves.** An optional extension models a 5-tick move as `start-move` (action) + `travel` (timer process) + `arrive` (event), since ENHSP does not support `:durative-action`; it shifts every feasibility boundary earlier without changing the lesson.
- **Scope.** Edges are assumed traversable (no obstacles / task–motion gap); single rover; deterministic (no sensor or energy uncertainty).